```
data = {
    'feature1': [1.0, 1.5, 3.0, 5.0, 3.5, 4.5, 3.5], 'feature2': [1.0, 2.0, 4.0, 7.0, 5.0, 5.0, 4.5]

                                                                                                    }

df = pd.DataFrame(data)

# Applying K-Means Clustering
kmeans = KMeans(n_clusters=2, random_state=42) kmeans.fit(df)

# Assigning Cluster Labels
df['cluster'] = kmeans.labels_
print(df)
```

Advanced Considerations

Choosing the Optimal Number of Clusters

Determining the optimal value for `k` is critical for effective clustering. Common techniques include:

1. **Elbow Method:** Plot the within-cluster sum of squares (WCSS) for different values of `k`. The "elbow" point, where the rate of decrease slows, suggests an appropriate value for `k`.

```
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42) kmeans.fit(df)
    wcss.append(kmeans.inertia_)

import matplotlib.pyplot as plt
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

2. **Silhouette Score:** Measures how similar a data point is to its own cluster compared to other clusters. Higher scores indicate better-defined clusters.

```
from sklearn.metrics import silhouette_score score = silhouette_score(df[['feature1', 'feature2']], kmeans.labels_) print("Silhouette Score:", score)
```

Preprocessing Data

Since K-Means relies on Euclidean distance, it is essential to preprocess the data: